Python shapefile processing workflow

The Python script acts as the bridge between a Survey123 attachment and the actual geometry used in the hosted feature layer and dashboard.

What problem it solves

In Survey123, a contractor can upload a zipped shapefile as an attachment, but that upload does not automatically become the feature geometry in ArcGIS Online. The attachment is just stored with the record. Without additional processing, the geometry in the hosted feature layer would still need to be drawn manually or updated by staff later.

The Python script solves that by reading the shapefile attachment after submission and writing its geometry directly into the hosted feature layer record.

What the script does

At a high level, the script does the following:

- Connects to ArcGIS using the authenticated local ArcGIS Pro / organization login session.
- Connects to the hosted feature layer using the feature layer REST URL.
- Queries records in the Fuel Treatments Submissions layer.
- Checks each record for attachments.
- Looks specifically for a zipped shapefile attachment.
- Downloads the attachment to a temporary local directory.
- Extracts the zip file.
- Finds the .shp file inside.
- Reads the shapefile into Python using geopandas.
- Merges the shapefile geometry if needed into a single geometry.
- Reprojects the geometry if necessary.
- Converts it into Esri-compatible JSON geometry.
- Updates the matching feature in the hosted feature layer using edit_features.
- Once the hosted layer is updated, the dashboard reflects the geometry automatically because it is already reading from that same feature layer.

The full chain is:

Survey123 submission	*Zipped shapefile attachment*	*Local Python script*	*Hosted feature layer geometry update*	*Dashboard update*

Why the script is run locally
The script is currently designed to run on a local machine, not inside Survey123 and not directly inside the dashboard.

That means:

- Survey123 collects the record and attachment
- Python runs separately afterward
- Python performs the geometry conversion and hosted layer update

This local-machine setup works well for testing and demonstration because it avoids needing ArcGIS Notebook privileges or server deployment.

Local machine setup
The script was run on a local Windows machine using the ArcGIS Pro Python environment.

Folder location

- The script was stored here:
  C:\Users\JXN_G\Desktop\summit_script

- The Python file was:
  process_shapefile_attachments.py

Python environment used
The script was not run in the default Windows terminal environment. It was run through the ArcGIS Pro Python environment because that environment includes the GIS-related packages needed for ArcGIS authentication and feature layer editing.

A cloned conda environment was created from the default ArcGIS Pro environment because the default environment is read-only.

Environment name

summit_env

This cloned environment allowed package installation, including geopandas.

Why the cloned environment was necessary
The default ArcGIS Pro environment, arcgispro-py3, is installed under Program Files and is not writable for normal package installation. When trying to install geopandas there, the install failed with an EnvironmentNotWritableError.

To fix that, a clone of the default environment was created:

```
conda create --name summit_env --clone arcgispro-py3
```

Then the cloned environment was activated:

```
conda activate summit_env
```

After that, geopandas was installed successfully in the cloned environment.

How the script is run locally

Each time the script needs to be run, the steps are:

1. *Open ArcGIS Python Command Prompt*
   This uses the ArcGIS Pro conda environment setup and avoids missing-package issues that would happen in a standard Windows command prompt.

2. Activate the cloned environment

```
conda activate summit_env
```

3. Navigate to the folder containing the script

```
cd C:\Users\JXN_G\Desktop\summit_script
```

4. Run the script

```
python process_shapefile_attachments.py
```

That is the full local execution workflow.

Authentication method used
Because the ArcGIS account is tied to the University of Utah organization login, standard username/password authentication was not the right approach.

Instead, the script uses:

```
GIS("home")
```

This tells the ArcGIS API for Python to use the authenticated ArcGIS Pro / organization session rather than hardcoded credentials.

That matters because the login uses organization-based authentication through the University of Utah, not a plain ArcGIS Online username/password workflow.

Feature layer connection

A major requirement was using the actual hosted feature layer REST URL, not the item page URL and not a web map URL.

The correct type of URL looks like this:
https://services1.arcgis.com/.../arcgis/rest/services/FuelTreatments_Submissions_/FeatureServer/0

Key point:

- **FeatureServer/0** points to the actual layer
- item page URLs like **home/item.html?id=...** do not work in the script

The script was configured against the Fuel Treatments Submissions layer, since that is the layer receiving the Survey123 records and feeding the operational dashboard.

What successful execution looked like
When the script processed a valid record, it returned output like:

```
OID 239: {'addResults': [], 'updateResults': [{'objectId': 239, 'uniqueId': 239, 'globalId': None, 'success': True}], 'deleteResults': []}
```

This means the hosted feature with ObjectID 239 was successfully updated.

That confirms the script was able to:

- locate the record
- process the shapefile attachment
- update the hosted feature geometry
- write the change back into ArcGIS Online successfully

What must exist for the script to work

For the script to actually process anything, the following must already exist:

1. A Survey123 submission in the hosted layer
2. A zipped shapefile uploaded as an attachment to that record
3. A valid hosted feature layer REST endpoint in the script
4. A functioning authenticated ArcGIS login session
5. The local Python environment with required packages installed

If no shapefile has been uploaded yet, the script can still run but may finish without visible output because there is nothing to process.

What the script makes possible

Using this code, the system can support the following:

1. **Attachment-to-geometry conversion**
   A contractor can upload a zipped shapefile, and that shapefile can become the actual hosted polygon geometry instead of remaining just an attachment.

2. **Centralized geometry storage**
   The final treatment geometry ends up stored in the hosted feature layer, which means the map and dashboard can use it directly.

3. **Reduced manual GIS editing**
   Without the script, someone would still need to manually open attachments, extract shapefiles, and update geometry by hand. The Python process reduces that backend GIS work substantially.

4. **Dashboard integration**
   Because the dashboard is already connected to the hosted feature layer, once the geometry is updated, the dashboard reflects the updated treatment polygon automatically.

5. **Support for two workflows**
   The form can support:
   - manual polygon drawing in Survey123
   - uploaded zipped shapefile processing through Python

   That gives flexibility depending on what the contractor has available.

What the script does not currently do automatically

At this stage, the script is manual, meaning someone still has to run it from the local machine.

So the workflow right now is:

- Submit Survey123 record
- Upload zipped shapefile
- Open ArcGIS Python Command Prompt
- Activate summit_env
- Navigate to the folder
- Run the script
- Verify the update

The script is not yet:

- triggered automatically at submission time
- running on a schedule
- deployed to a server
- running inside ArcGIS Online notebooks

What could be added later

This code could be extended in several ways.

1. **Scheduled automation**
   Instead of running manually, it could be run on a schedule from:
   - ArcGIS Notebooks, if enabled
   - Windows Task Scheduler on a local or shared machine
   - a server or VM

2. **Better logging**
   The script could write out:
   - which ObjectIDs were processed
   - which attachments failed
   - why a shapefile was rejected

3. **Status fields**
   The hosted layer could include fields such as:
   - ProcessingStatus
   - GeometrySource
   - ProcessedDate

   That would make it easier to track what has already been processed to avoid duplicates.

4. **Validation**
   The script could check:
   - whether the uploaded zip actually contains a shapefile
   - whether the shapefile has polygon geometry
   - whether there are multiple features
   - whether the coordinate system is valid

5. **Acreage recalculation**
   The script can also be expanded to recalculate acreage after the geometry update so the hosted layer always reflects the final uploaded boundary.

Practical meaning for the workflow

The Python script demonstrates that uploaded shapefile attachments do not have to remain passive files in Survey123. They can be actively used to populate and update the operational GIS layer.

That means the overall system can move closer to:

- one intake form
- one hosted dataset
- one dashboard source
- less manual data handling
- more consistent geometry across submissions

The current version proves that this is feasible on a local machine using ArcGIS Pro's Python environment and an organization-authenticated ArcGIS connection.

Local run instructions summary

For reference, the local-machine execution process is:

```
conda activate summit_env
```

```
cd C:\Users\JXN_G\Desktop\summit_script
```

```
python process_shapefile_attachments.py
```

That is the exact command-line workflow used to run the script locally.

Final takeaway

The code provides a working proof of concept for converting uploaded zipped shapefiles into hosted feature geometry in ArcGIS Online using a local Python process. It does not rely on manual geometry editing after submission, and it shows that Survey123 attachments can be turned into dashboard-ready GIS data when paired with Python automation.